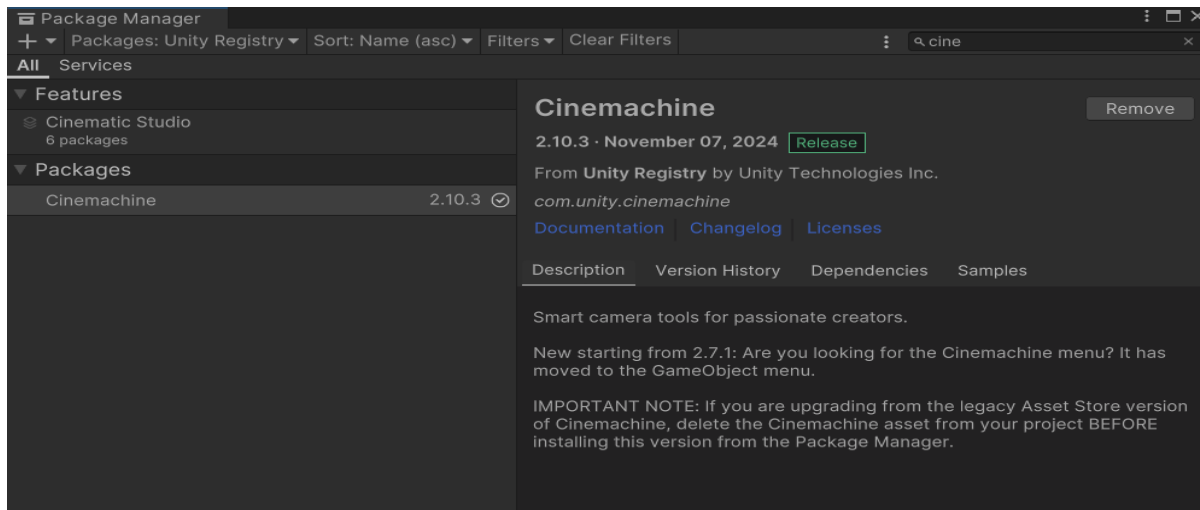


Lab Task: Exploring Unity Cinemachine and Its Properties

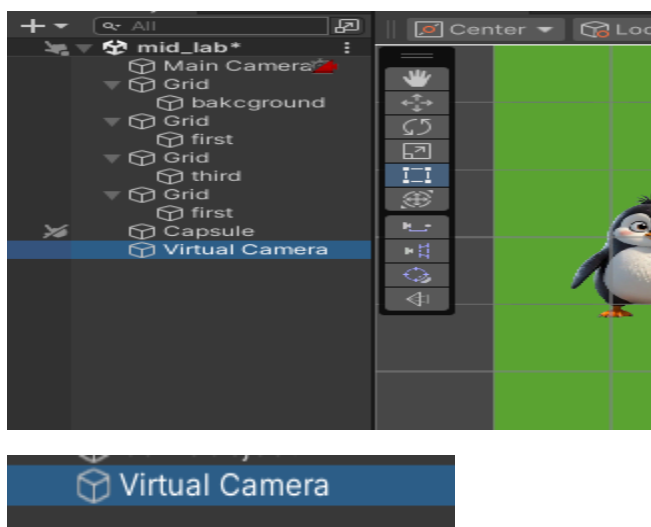
1. Setting Up the Project

1. Create a New Project:
2. Install Cinemachine:



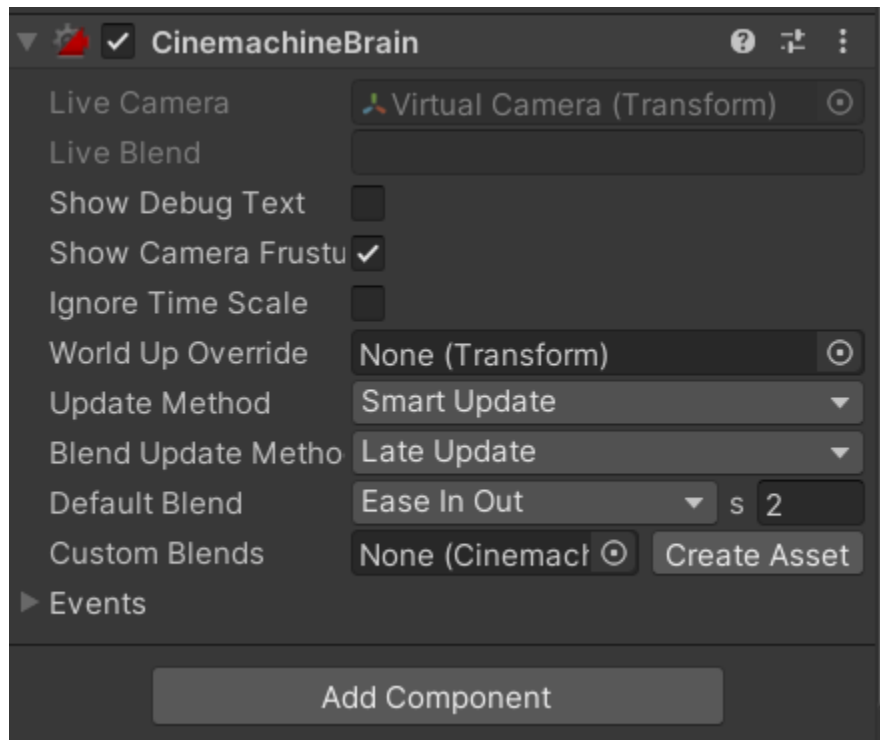
2. Adding Cinemachine to the Scene

1. Add a Virtual Camera:



2. Observe the Main Camera:

- Unity will automatically attach a **Cinemachine Brain** to the Main Camera.



Lab Task: Exploring Unity Cinemachine and Its Properties for 2D Games

This task will help you set up and explore Unity's Cinemachine to create dynamic camera systems tailored for 2D games. Follow the steps below to implement a basic camera system, customize its behavior, and add cinematic effects.

Objective

- Understand and utilize Cinemachine components like the **Cinemachine Virtual Camera** and **Cinemachine Brain**.
 - Learn to configure camera properties such as Follow, Lens, Body, and Aim.
 - Create dynamic camera transitions and add cinematic effects to enhance gameplay.
-

Task Breakdown

1. Setting Up the Project

1. **Create a New Project:**
 - Open Unity and create a new **2D project**.
 2. **Install Cinemachine:**
 - Go to Window > Package Manager.
 - Search for **Cinemachine** and click **Install**.
-

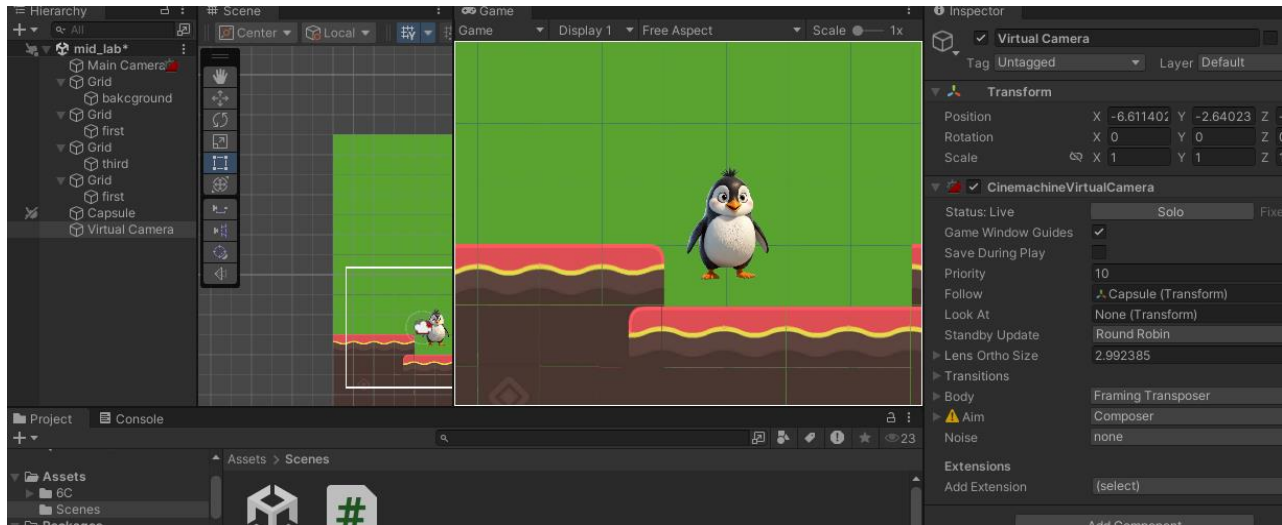
2. Adding Cinemachine to the Scene

1. **Add a Virtual Camera:**
 - In the **Hierarchy**, right-click and select Cinemachine > Create 2D Camera.
 - This will create a **Cinemachine Virtual Camera** in your scene.
 2. **Observe the Main Camera:**
 - Unity will automatically attach a **Cinemachine Brain** to the Main Camera.
-

3. Exploring Cinemachine Virtual Camera Properties

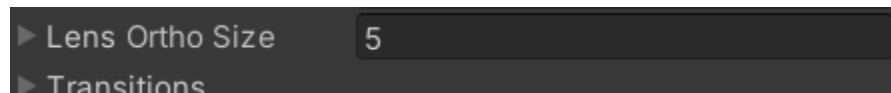
1. **Follow:**

- Assign the player GameObject (e.g., a sprite with movement controls) to the **Follow** field.



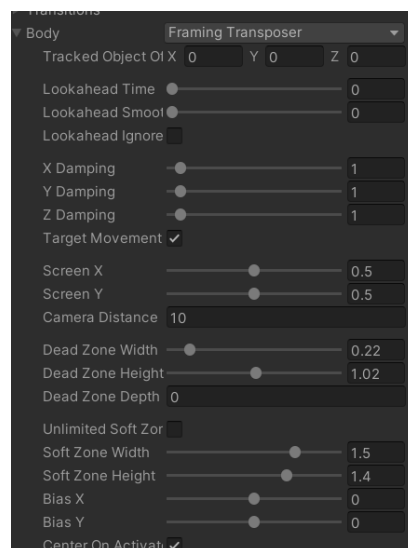
2. Lens:

- Adjust the **Orthographic Size** to control how much of the scene is visible.



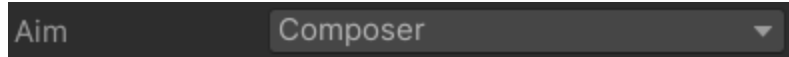
3. Body:

- Use **Framing Transposer** for smooth following of the player:
 - Adjust the **Dead Zone** and **Soft Zone** to refine how the camera responds to player movement.



4. Aim:

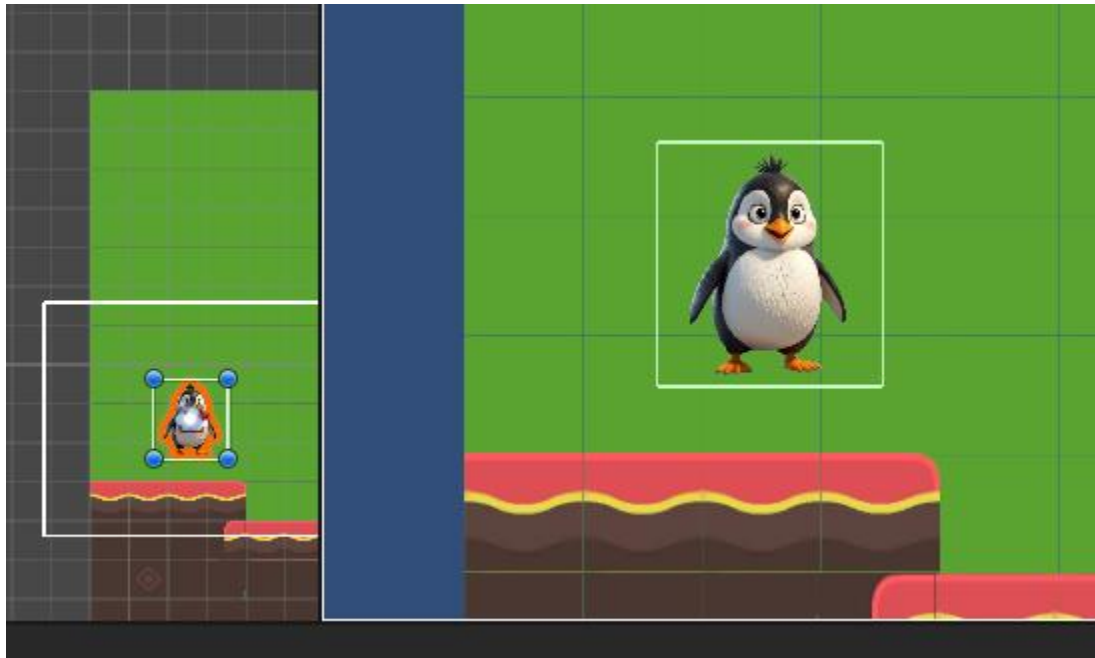
- Use **Composer** to adjust the camera's focus on the player dynamically.



4. Implementing a Basic Camera Follow System

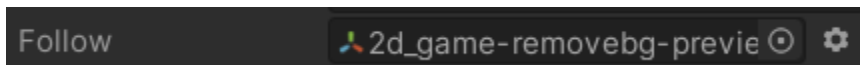
1. Create a Player:

- Add a 2D sprite (e.g., a character) to your scene.
- Add a Rigidbody2D and movement script to control the player:



Assign the Player to Follow:

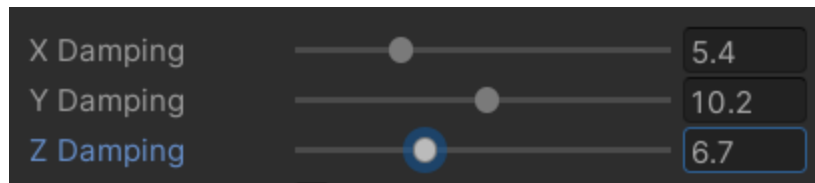
- Drag the player GameObject to the **Follow** property of the Virtual Camera.



5. Customizing Camera Behavior

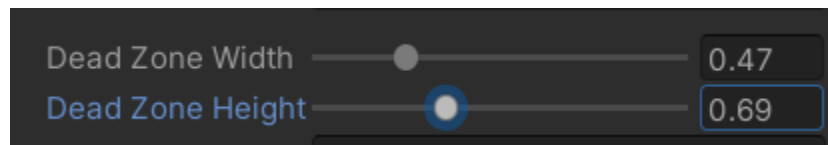
1. Damping:

- Increase the **Damping** values in the Framing Transposer to make the camera's movement smoother.



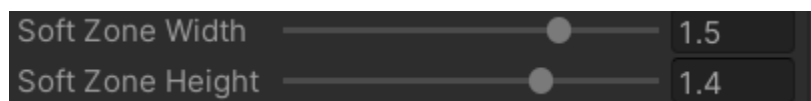
2. Dead Zone:

- Define a **Dead Zone** where the camera won't move when the player is inside it.



3. Soft Zone:

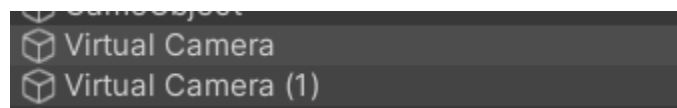
- Create a **Soft Zone** to make the camera movement more gradual when the player exits the Dead Zone.



6. Switching Between Cameras

1. Add a Second Virtual Camera:

- Right-click in the **Hierarchy** and create another **2D Camera**.



- Configure this camera to focus on a different part of the scene or provide a zoomed-in perspective.



2. Write a Camera Switch Script:

```

using UnityEngine;

public class CameraSwitcher : MonoBehaviour
{
    public Cinemachine.CinemachineVirtualCamera cam1;
    public Cinemachine.CinemachineVirtualCamera cam2;

    void Update()
    {
        if (Input.GetKeyDown(KeyCode.Alpha1))
        {
            cam1.Priority = 10;
            cam2.Priority = 0;
        }
        else if (Input.GetKeyDown(KeyCode.Alpha2))
        {
            cam1.Priority = 0;
            cam2.Priority = 10;
        }
    }
}

```

Camera Shake:

- Select a Virtual Camera and expand the **Noise** section in the Inspector.
- Choose a **Basic Multi-Channel Perlin** noise profile.



- Adjust the amplitude and frequency to simulate camera shake during events like explosions or impacts.

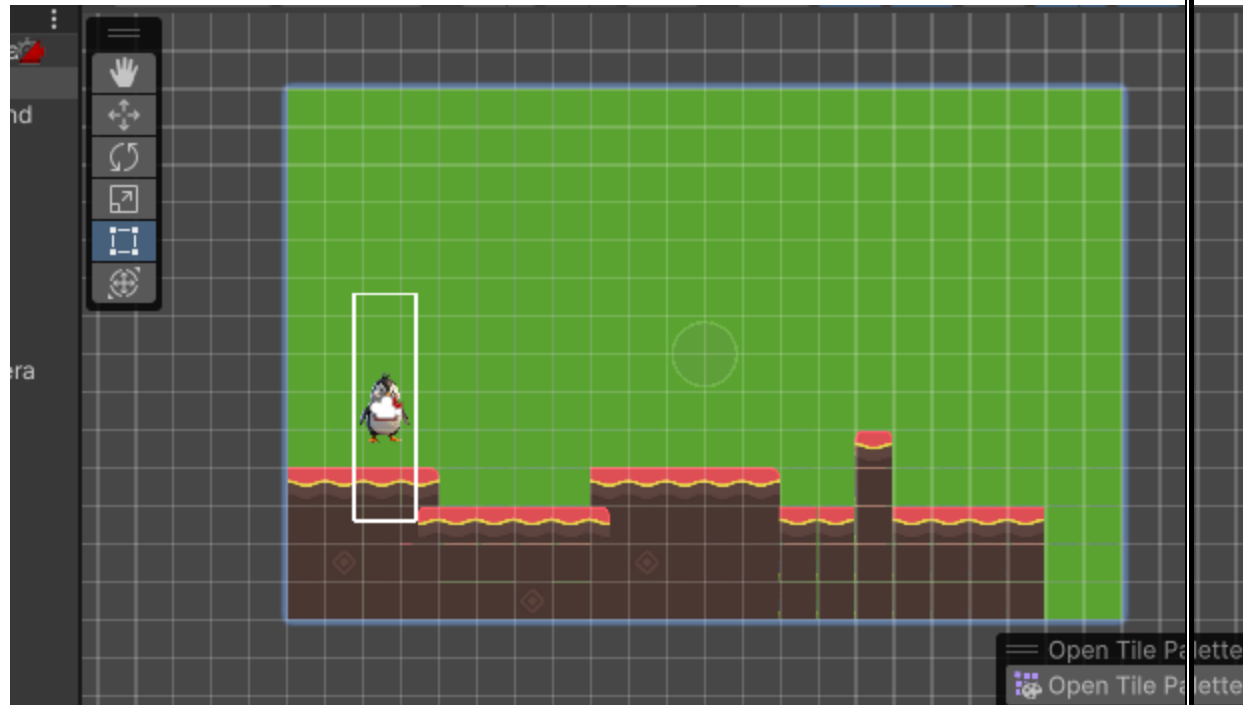


Lab Task: Exploring Grid Tilemap and Tile Palette in Unity

1. Create a Tilemap

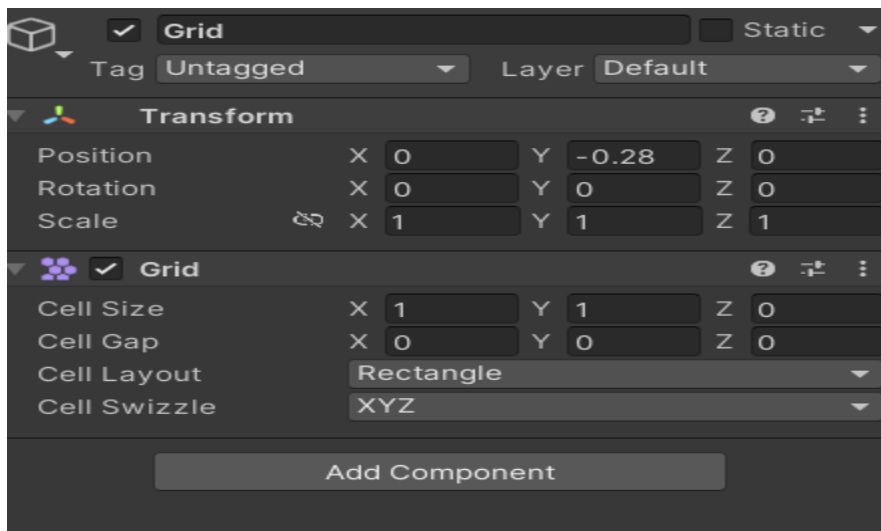
1. Create a Tilemap:

- In the **Hierarchy**, right-click and select 2D Object > Tilemap > Rectangular.
- Unity will automatically create a **Grid** parent object and a **Tilemap** child object.



2.Explore Grid Properties:

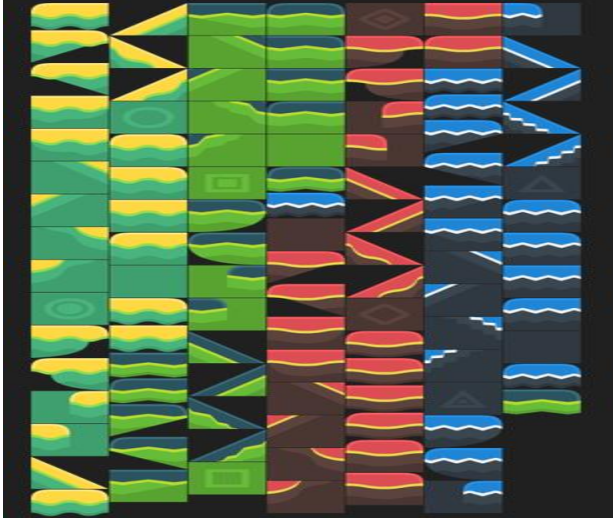
- Select the **Grid** object in the **Hierarchy**.
- In the **Inspector**, explore properties like **Cell Size** (to scale the tiles) and **Cell Layout** (e.g., **Rectangular**, **Hexagonal**, etc.).



3. Import Tile Sprites

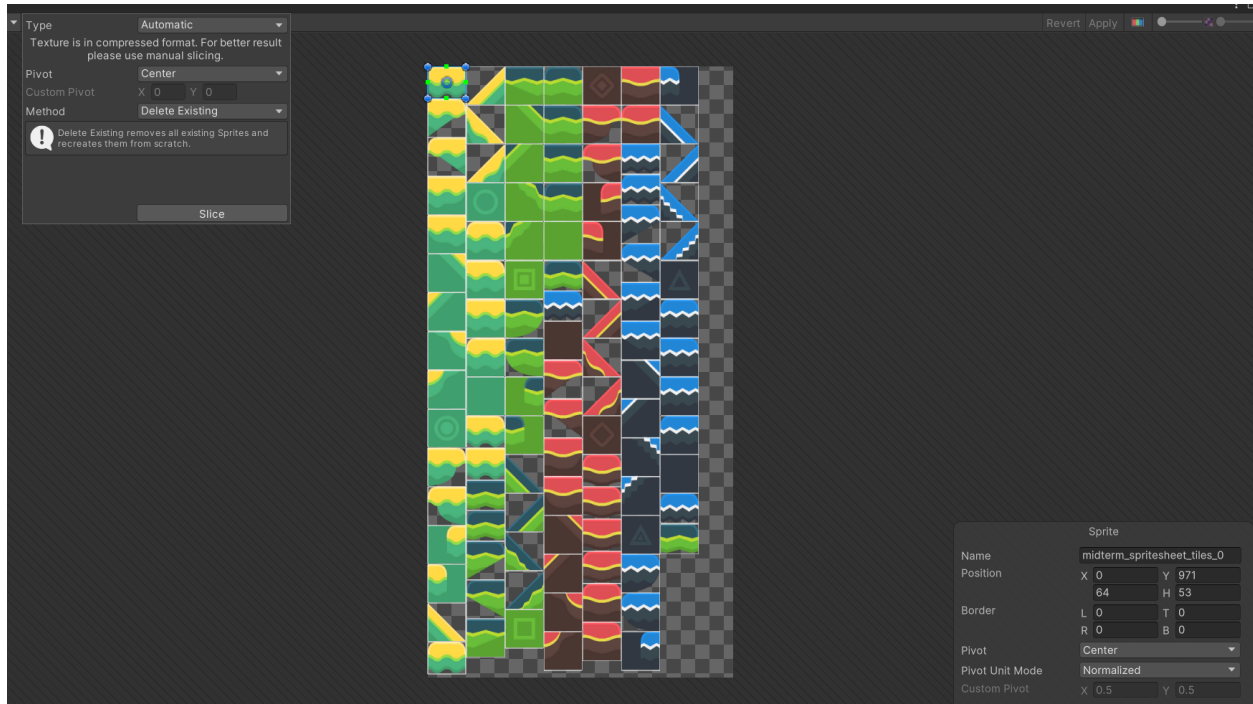
1. Download a Tile Sprite Sheet:

- Download a sprite sheet with multiple tile designs (e.g., grass, water, stone, etc.).



2. Slice the Sprite Sheet:

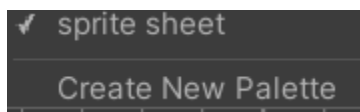
- In the **Project** window, select the sprite sheet and click **Sprite Editor**.
- Slice the sheet into individual sprites (one for each tile) and click **Apply**.



4. Create a Tile Palette

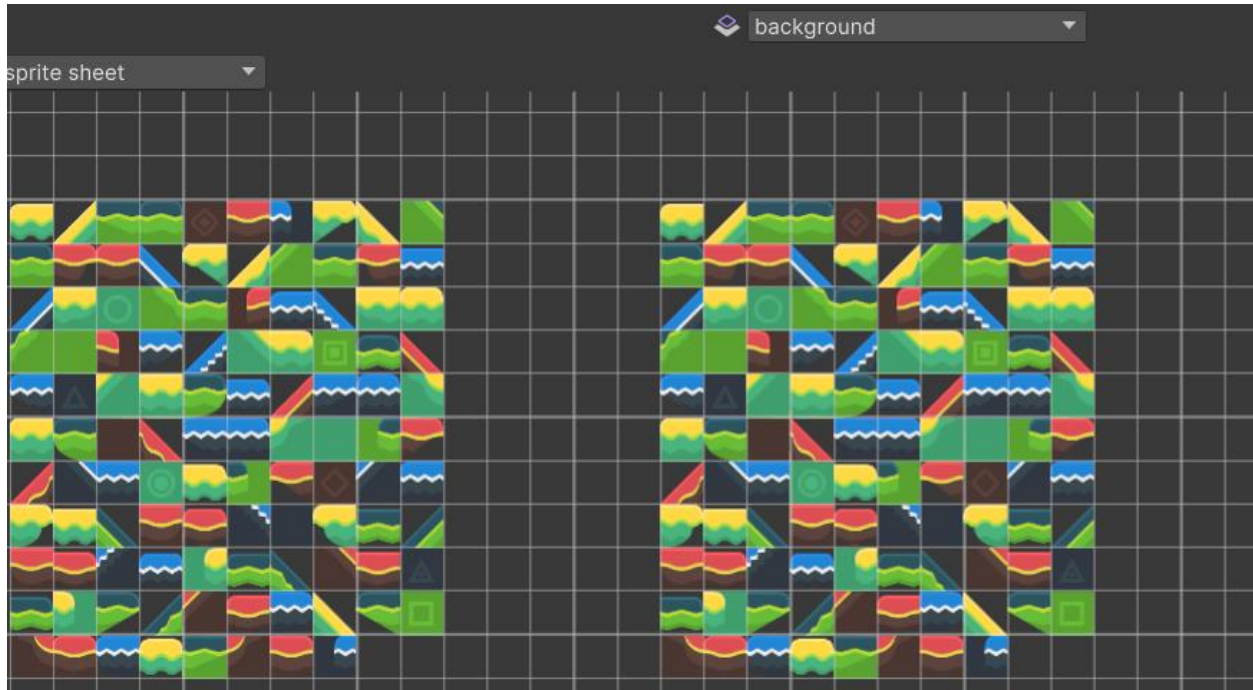
1. Open the Tile Palette:

- Go to Window > 2D > Tile Palette.
- Create a new Tile Palette and name it **EnvironmentPalette**.



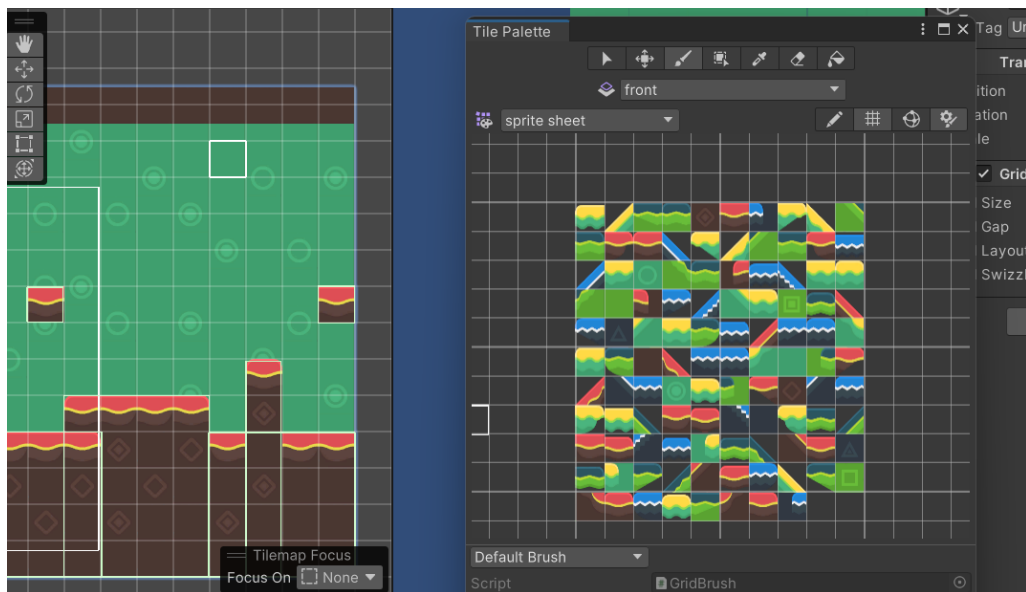
2. Drag Sprites into the Palette:

- Drag the sliced tile sprites from the **Project** window into the **Tile Palette** window.
- Unity will automatically create **Tile** assets for each sprite.



5. Painting the Tilemap

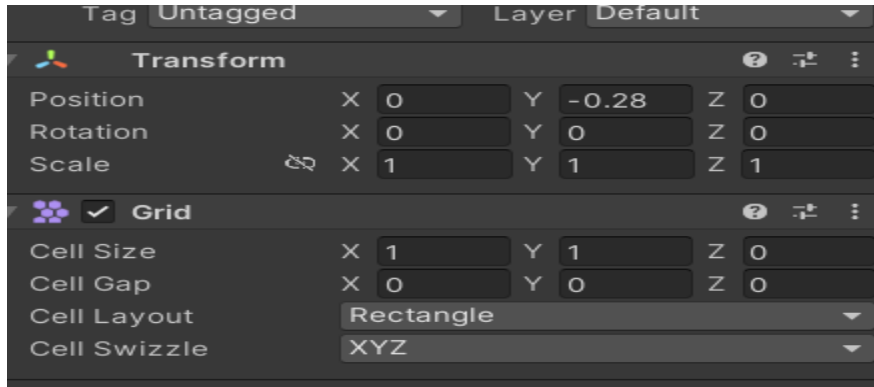
1. Use the Paintbrush Tool:



6. Experiment with Grid and Tilemap Properties

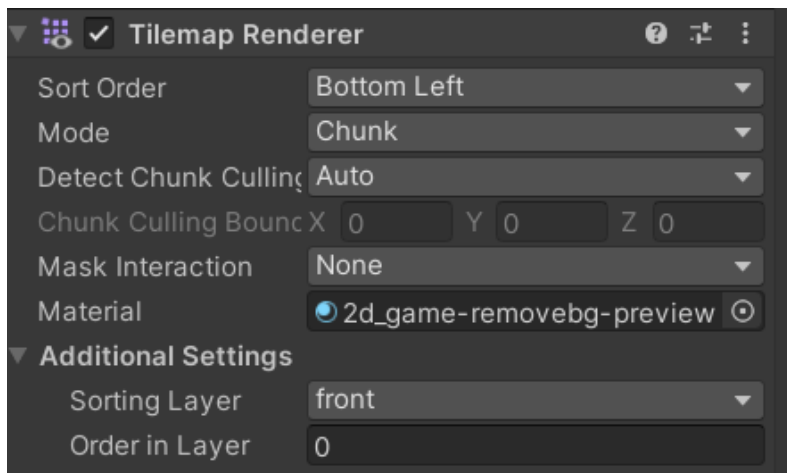
1. Modify Grid Properties:

- Select the **Grid** object and experiment with the following properties:
 - **Cell Layout:** Change between **Rectangular** and **Isometric** grids.
 - **Cell Size:** Modify the size of the tiles and observe how they adjust on the grid.



2. Modify Tilemap Renderer:

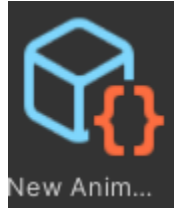
- Select the **Tilemap** object in the **Hierarchy**.
- Explore the **Tilemap Renderer** component:



7. Adding Custom Tile Properties

1. Create a Scriptable Tile:

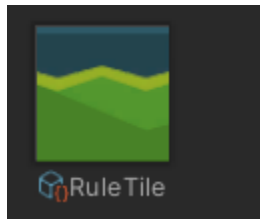
- Right-click in the **Assets** folder and select Create > 2D > Tile.
- Assign a sprite to the new tile and modify its settings in the **Inspector** (e.g., adding a **color tint**, setting a **collider type**).



8. Rule Tiling

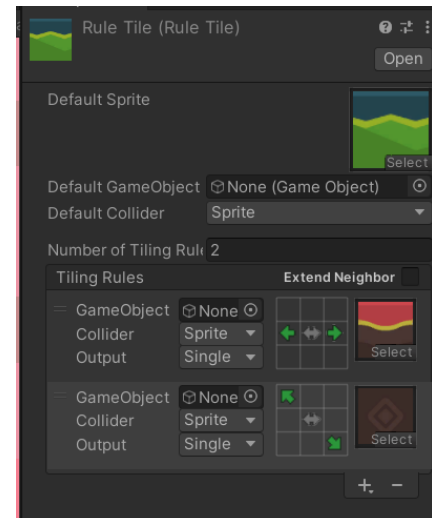
1. Create a Rule Tile:

- Right-click in the **Assets** folder and choose Create > 2D > Tile > Rule Tile.
- Create rules for dynamic terrain transitions (e.g., auto-connecting paths or edges).



2. Apply the Rule Tile:

- Drag and drop the **Rule Tile** onto the **Tilemap**.
- Experiment with how the Rule Tile automatically arranges tiles based on neighboring tiles.



9. Add a Player Character

- Create a simple 2D character with basic movement scripting.
- Test movement across different grid layouts and terrains.

